Architecture Design

System architecture design translates your requirements into a comprehensive technical blueprint. This section guides you through creating detailed architecture specifications with visual diagrams, component definitions, and integration patterns.

Exercise 4.1: High-Level Architecture Design

Create the overall system architecture using AWS serverless best practices:

```
Design the high-level architecture for our serverless MyCoolChatbot-KiroSpec application using AWS services:

**Architecture Requirements**:
Based on our functional and non-functional requirements, design a serverless architecture that includes:

**Frontend Layer**:
- Static website hosting with global CDN
- Progressive Web App capabilities
- Responsive design for all devices
- Offline functionality and caching

**API Layer**:
- RESTful API with proper versioning
- Authentication and authorization
- Rate limiting and throttling
- Request/response validation

**Compute Layer**:
- Serverless functions for business logic
- Auto-scaling and cost optimization
- Error handling and retry logic
- Integration with AI services

**Data Layer**:
- User data and session management
- Conversation history storage
- Configuration and settings
- Audit logging and compliance

**AI Integration Layer**:
- Amazon Bedrock integration
- Model selection and optimization
- Prompt engineering and response processing
- Cost monitoring and optimization

**Security Layer**:
- Identity and access management
- Data encryption and protection
- Network security and isolation
- Compliance and audit logging

Use the AWS documentation MCP server to research current serverless architecture patterns and incorporate AWS Well-Architected Framework principles into your de

Create a high-level architecture diagram showing all major components and their relationships.
```

Expected Outcome

- Comprehensive high-level architecture aligned with requirements
- Clear component separation and responsibility definition
- Integration of AWS Well-Architected Framework principles
- Visual architecture diagram with component relationships
- Justification for architectural decisions based on requirements

Exercise 4.2: Detailed Component Architecture

Design detailed specifications for each architectural component:

```
Create detailed component specifications for each layer of the architecture:

**Frontend Components**:
- **Static Website (S3 + CloudFront)**:
  - Hosting configuration and optimization
  - CDN distribution and caching strategies
  - Security headers and HTTPS enforcement
  - Custom domain and SSL certificate management

- **Web Application (React + TypeScript)**:
  - Component hierarchy and state management
  - Routing and navigation patterns
  - API integration and error handling
  - Progressive Web App implementation

**API Gateway Components**:
- **REST API Configuration**:
  - Endpoint design and resource organization
  - Request/response models and validation
  - CORS configuration and security headers
  - Usage plans and API key management

- **Authentication Integration**:
  - JWT token validation and processing
  - Authorization policies and role-based access
  - Session management and timeout handling
  - Multi-factor authentication support

**Lambda Function Components**:
- **Chat Processing Function**:
  - Message validation and sanitization
  - Bedrock integration and response processing
  - Conversation context management
  - Error handling and retry logic

- **Authentication Function**:
  - User registration and login processing
  - Password validation and security
  - JWT token generation and validation
  - Session management and cleanup

- **User Management Function**:
  - Profile management and preferences
  - Data privacy and GDPR compliance
  - Account deactivation and data deletion
  - Audit logging and compliance tracking

For each component, specify:
- Detailed functionality and responsibilities
- Input/output interfaces and data models
- Error handling and recovery procedures
- Performance and scalability requirements
- Security and compliance considerations

Use the AWS documentation MCP server to research best practices for each AWS service and incorporate current recommendations.
```

Expected Outcome

- Detailed component specifications with clear responsibilities
- Well-defined interfaces and data models for all components
- Comprehensive error handling and recovery strategies
- Performance and scalability considerations for each component
- Security and compliance requirements integrated throughout

Exercise 4.3: Data Flow and Integration Patterns

Design comprehensive data flows and integration patterns:

```
Create detailed data flow diagrams and integration patterns for all system interactions:

**User Interaction Flows**:
1. **User Registration Flow**:
   - Frontend form validation and submission
   - API Gateway request routing and validation
   - Lambda function processing and validation
   - User data storage and confirmation
   - Email verification and account activation

2. **Authentication Flow**:
   - Login credential validation
   - JWT token generation and response
   - Session establishment and management
   - Token refresh and expiration handling
   - Logout and session cleanup

3. **Chat Message Flow**:
   - Message input and frontend validation
   - API request with authentication
   - Lambda function message processing
   - Bedrock API integration and response
   - Response processing and frontend display

**Data Integration Patterns**:
- **Synchronous Patterns**: Real-time API calls and responses
- **Asynchronous Patterns**: Event-driven processing and notifications
- **Batch Processing**: Bulk operations and data migration
- **Error Handling**: Retry logic, circuit breakers, and fallback mechanisms
- **Monitoring Integration**: Logging, metrics, and alerting throughout all flows

**External Service Integration**:
- **Amazon Bedrock Integration**:
  - Model selection and configuration
  - Request formatting and response processing
  - Error handling and fallback strategies
  - Cost optimization and usage monitoring

- **AWS Services Integration**:
  - CloudWatch logging and monitoring
  - Secrets Manager for configuration
  - S3 for file storage and static assets
  - CloudFormation/CDK for infrastructure management

Create detailed sequence diagrams for each major user flow and integration pattern, showing all system interactions, error conditions, and recovery procedures.
```

Expected Outcome

- Comprehensive data flow diagrams for all user interactions
- Detailed integration patterns for internal and external services
- Sequence diagrams showing system interactions and timing
- Error handling and recovery procedures for all flows
- Performance and scalability considerations for data processing

Exercise 4.4: Security Architecture Design

Design comprehensive security architecture and controls:

```
Create a detailed security architecture that addresses all security requirements:

**Security Layers and Controls**:

**Network Security**:
- CloudFront security headers and WAF integration
- API Gateway security policies and rate limiting
- VPC configuration and network isolation (if needed)
- Security groups and network access controls

**Identity and Access Management**:
- IAM roles and policies with least privilege
- Service-to-service authentication patterns
- User authentication and authorization flows
- Multi-factor authentication implementation

**Data Protection**:
- Encryption at rest for all data storage
- Encryption in transit for all communications
- Key management and rotation policies
- Data classification and handling procedures

**Application Security**:
- Input validation and sanitization
- Output encoding and XSS prevention
- SQL injection and command injection prevention
- CSRF protection and secure session management

**Monitoring and Incident Response**:
- Security event logging and monitoring
- Automated threat detection and response
- Incident response procedures and escalation
- Forensics and investigation capabilities

**Compliance and Governance**:
- GDPR compliance implementation
- SOC 2 controls and audit procedures
- Data retention and deletion policies
- Privacy by design implementation

Create a security architecture diagram showing all security controls, data flows, and trust boundaries. Include threat modeling and risk assessment for each com

Use the AWS documentation MCP server to research current AWS security best practices and incorporate them into your security architecture.
```

Expected Outcome

- Comprehensive security architecture with layered defense
- Detailed security controls for each architectural component
- Threat modeling and risk assessment documentation
- Compliance framework implementation
- Security monitoring and incident response procedures

Exercise 4.5: Deployment and Operations Architecture

Design deployment and operational architecture:

```
Create comprehensive deployment and operations architecture:

**Deployment Architecture**:
- **Infrastructure as Code**: AWS CDK stack organization and deployment
- **CI/CD Pipeline**: Build, test, and deployment automation
- **Environment Management**: Dev, test, staging, and production environments
- **Blue/Green Deployment**: Zero-downtime deployment strategies
- **Rollback Procedures**: Automated and manual rollback capabilities

**Operations Architecture**:
- **Monitoring and Observability**: CloudWatch, X-Ray, and custom metrics
- **Logging Strategy**: Centralized logging and log analysis
- **Alerting and Notifications**: Proactive monitoring and incident response
- **Performance Monitoring**: Application and infrastructure performance tracking
- **Cost Monitoring**: Resource usage and cost optimization

**Scalability and Performance**:
- **Auto-scaling Policies**: Lambda concurrency and API Gateway throttling
- **Caching Strategies**: CloudFront, API Gateway, and application-level caching
- **Database Optimization**: Query optimization and connection management
- **Content Delivery**: Global CDN and edge optimization

**Disaster Recovery and Business Continuity**:
- **Backup Strategies**: Data backup and recovery procedures
- **Multi-Region Deployment**: High availability and disaster recovery
- **Failover Procedures**: Automated and manual failover capabilities
- **Recovery Testing**: Regular disaster recovery testing and validation

**Operational Procedures**:
- **Maintenance Windows**: Scheduled maintenance and updates
- **Capacity Planning**: Resource planning and scaling strategies
- **Incident Management**: Incident response and resolution procedures
- **Change Management**: Change control and approval processes

Create deployment diagrams showing CI/CD pipelines, environment promotion, and operational monitoring architecture.

Research current AWS operational best practices using the MCP servers and incorporate them into your operational architecture.
```

Expected Outcome

- Comprehensive deployment architecture with CI/CD automation
- Detailed operational procedures and monitoring strategies
- Scalability and performance optimization plans
- Disaster recovery and business continuity procedures
- Operational excellence framework following AWS best practices

Exercise 4.6: Architecture Validation and Documentation

Validate architecture against requirements and create comprehensive documentation:

```
Validate your architecture design and create comprehensive documentation:

**Architecture Validation**:
1. **Requirements Traceability**: Map each architectural component to specific requirements
2. **Performance Validation**: Verify architecture meets all performance requirements
3. **Security Validation**: Ensure all security requirements are addressed
4. **Scalability Validation**: Confirm architecture supports scalability requirements
5. **Cost Validation**: Estimate costs and validate against budget constraints

**Architecture Documentation**:
- **Architecture Decision Records (ADRs)**: Document key architectural decisions and rationale
- **Component Specifications**: Detailed specifications for each architectural component
- **Interface Definitions**: API contracts and data models for all integrations
- **Deployment Guides**: Step-by-step deployment and configuration procedures
- **Operational Runbooks**: Procedures for monitoring, maintenance, and troubleshooting

**Architecture Review Process**:
- **Stakeholder Review**: Present architecture to stakeholders for feedback and approval
- **Technical Review**: Peer review of architectural decisions and implementations
- **Security Review**: Security assessment and threat modeling validation
- **Performance Review**: Performance analysis and optimization recommendations
- **Cost Review**: Cost analysis and optimization opportunities

**Architecture Governance**:
- **Change Control**: Process for architectural changes and impact assessment
- **Version Control**: Versioning strategy for architecture documentation
- **Compliance Validation**: Ensure architecture meets all compliance requirements
- **Continuous Improvement**: Process for architecture evolution and optimization

Set up Agent Hooks to automatically validate architecture documentation completeness and consistency when architecture files are updated.

Create a comprehensive architecture specification document that serves as the blueprint for all subsequent technical specifications and implementation work.
```

Expected Outcome

- Validated architecture that fully addresses all requirements
- Comprehensive architecture documentation and specifications
- Architecture Decision Records documenting key decisions and rationale
- Architecture review and governance processes
- Automated validation through Agent Hooks integration

Key Deliverables

After completing the Architecture Design exercises, you should have:

1. **High-Level Architecture**: Overall system design with component relationships
2. **Detailed Component Specifications**: Comprehensive specifications for each component
3. **Data Flow and Integration Patterns**: Detailed interaction and integration designs
4. **Security Architecture**: Comprehensive security controls and procedures
5. **Deployment and Operations Architecture**: Complete operational framework
6. **Architecture Documentation**: Comprehensive documentation and governance

Integration with Advanced Features

Your architecture design process leverages:

- **MCP Servers**: For researching current AWS best practices and patterns
- **Agent Steering**: For applying consistent architectural standards and patterns
- **Agent Hooks**: For automated validation and documentation maintenance

Foundation Phase Completion

With the Foundation phase complete, you have established:

- **Project Structure**: Well-organized project with proper tooling
- **Advanced Features**: MCP servers, Agent Hooks, and Agent Steering configured
- **Comprehensive Requirements**: Detailed functional and non-functional requirements
- **System Architecture**: Complete architectural blueprint for implementation

Next Steps

Your foundation specifications provide the blueprint for the detailed technical specifications that follow. The next phases will focus on:

1. **Infrastructure Specifications**: Detailed AWS resource configurations
2. **Frontend Specifications**: UI/UX and technical implementation details
3. **Backend Specifications**: API design and business logic implementation
4. **Security Specifications**: Comprehensive security controls and compliance

Each phase will build upon your foundation specifications, using the advanced Kiro features you've configured to maintain consistency and quality throughout the development process.

Exercise 4.7: Foundation Implementation

Implement the foundation specifications using Kiro's advanced features:

````
Based on all foundation specifications, implement the project foundation:

**Step 1: Project Structure Implementation**
Using the project setup specifications, create the complete project structure:

```bash
# Create the project structure as specified
mkdir -p MyCoolChatbotSpec/{.kiro/{settings,steering},specifications/{foundation,infrastructure,frontend,backend,security},implementation/{infrastructure,fronte

# Initialize package.json with dependencies from specifications
npm init -y
npm install --save-dev typescript @types/node eslint prettier jest

# Create configuration files as specified
touch tsconfig.json .eslintrc.js .prettierrc jest.config.js
```

**Step 2: Advanced Features Implementation**
Implement all MCP servers, Agent Hooks, and Agent Steering as specified:

1. Configure MCP servers in `.kiro/settings/mcp.json` with all specified servers
2. Create Agent Hooks for specification validation and documentation sync
3. Implement Agent Steering files with project standards and security requirements
4. Test all advanced features integration as specified

**Step 3: Requirements Implementation**
Convert requirements specifications into actionable development tasks:

1. Create detailed user stories from functional requirements
2. Define acceptance criteria for each requirement
3. Set up requirements traceability matrix
4. Implement requirements validation processes

**Step 4: Architecture Implementation**
Create architecture artifacts from design specifications:

1. Generate architecture diagrams using AWS documentation MCP server
2. Create component specifications and interface definitions
3. Document deployment architecture and operational procedures
4. Validate architecture against requirements

**Validation Checklist**:
- [ ] All MCP servers configured and tested
- [ ] Agent Hooks created and functioning
- [ ] Agent Steering files implemented and active
- [ ] Requirements documented with acceptance criteria
- [ ] Architecture diagrams and documentation complete
- [ ] Project structure matches specifications exactly
- [ ] All configuration files created and validated

Use Agent Hooks to automatically validate the implementation against specifications and update documentation as changes are made.
````


- Comprehensive requirements and architecture documentation
- Validated project structure ready for development
- Automated quality assurance through Agent Hooks and Steering